



Software Security

Lecture #08

The Nature of Software

The following aspects of software create special challenges

1. Easily changeable
2. Easily reproducible
3. Hard to specify
4. Hard to ensure there are no bugs

Softwarizing a System

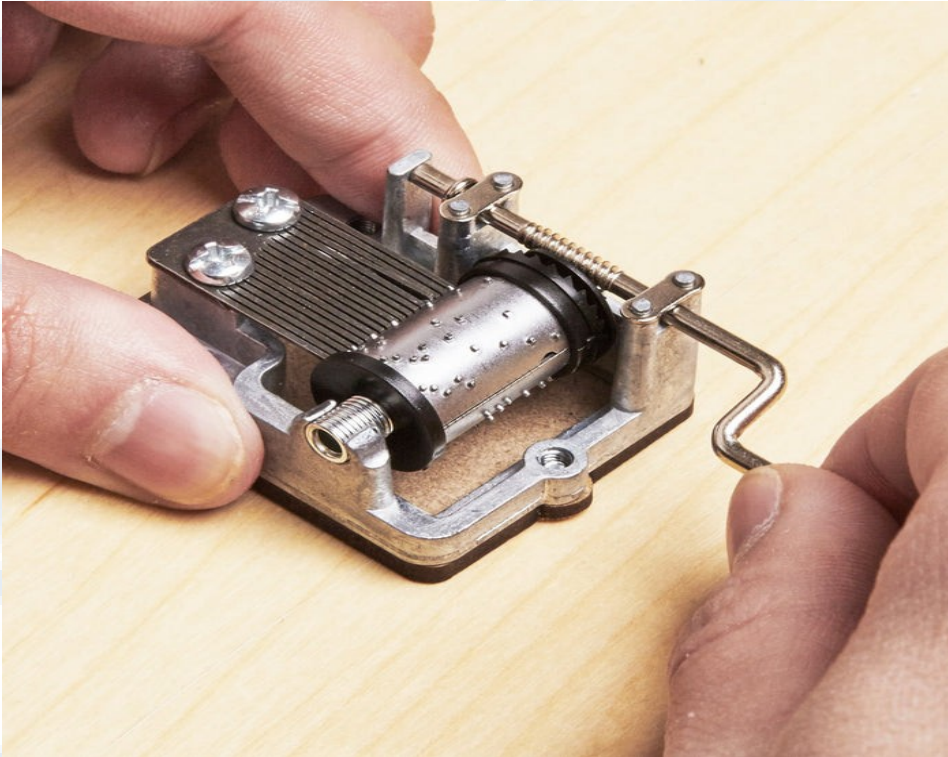
This refers to

- Replacing parts of a system with software running on a computer or using computer running on a system to simulate an existing physical system.
- The software is then considered to ***be the system*** and the behaviour of this software is considered to actually reflect what would happen in the real system

Some examples are

- Aircraft simulators
- Machine learning based system to aid/replace judges

Information processing in systems



In conventional systems the information content and information processing is integrally a part of the whole system and can only be understood by an understanding of that system



The concept of software has allowed us to abstract the Information content and information processing components away from the “physical” part of the system

Challenges arising from Softwarization

- The *information model* of the system which forms the basis for the software is often neither complete nor correct.
 - Some standards exist but these are not in very wide use
 - Very few Computer/IT engineers are trained on these standards
 - Software Engineering as a discipline is not a compulsory requirement
 - Very few (almost none) of the large software development projects tendered by the Government of India mandate the use of these standards
- The software should be a *complete and correct* implementation of the information model. These issues are addressed by the discipline of software engineering
 - Very few Computer/IT engineers are trained in this area
 - Software Engineering as a discipline is not a compulsory requirement

The basis for trustworthiness in software systems

Technical

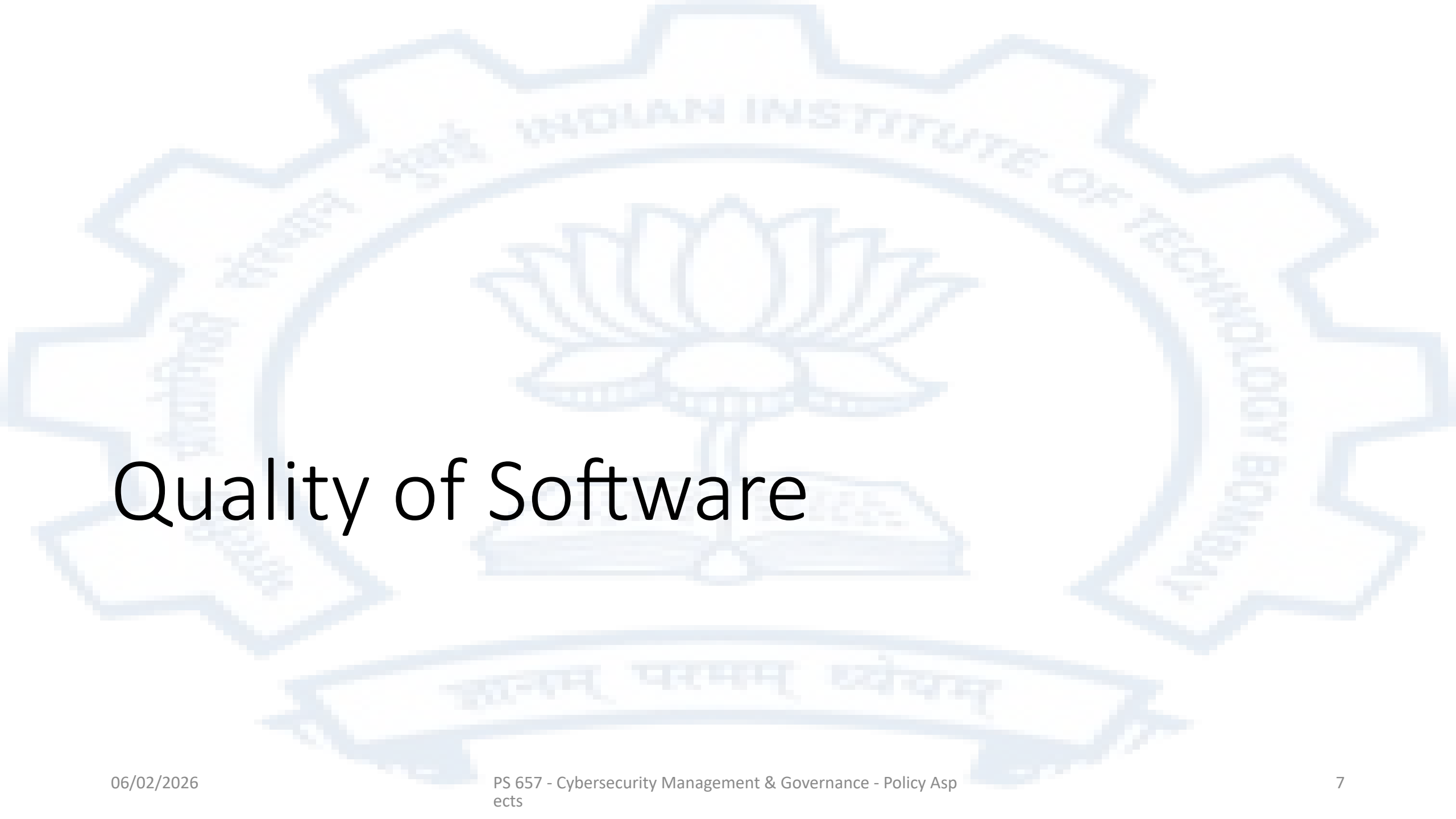
- Quality – Complexity and rapid change make it hard to adhere to specifications
- Safety and Security – Complexity and ease of modification lead to integrity issues and security failures can cause financial loss and physical harm. Penetrate and patch philosophy by software providers
- Problem/Incident/Crisis management

Risk Transference

- Insurance – lack of data valuation methods and actuarial data make it very hard to get meaningful insurance

Legal

- Product liability – No warranties. Software producers have stoutly resisted any type of product liability
- Jurisdiction – The use of the Internet as an essential element of software based systems has created serious jurisdiction challenges

The background of the slide features a large, light blue watermark of the Indian Institute of Technology Bombay (IIT Bombay) logo. The logo is a gear-like shape with a lotus flower in the center. The text "INDIAN INSTITUTE OF TECHNOLOGY BOMBAY" is written along the top inner edge of the gear, and "सत्यम् धर्मम् कर्मणम्" is written along the bottom inner edge.

Quality of Software

Aspects of Software Quality

- Functional aspects
 - Does the software correctly perform its specified functions
 - Adaptability and Extensibility – can new features be added easily
- Non-functional
 - Usability
 - Interfaces, navigability, accessibility
 - Performance
 - Efficiency, resource adequacy, scalability
 - Security
 - Confidentiality, availability, integrity, authentication, authorization, vulnerability management
 - Supportability
 - Technology debt avoidance, modularity

Software specification

- Specification of requirements (What is needed and not how it is to be implemented)
 - User requirements
 - Operations requirements
 - Support requirements
 - System requirements
- Design
 - How is the system to be modeled
 - How the requirements will be met
- Code
 - Realization of the code in a formal programming language

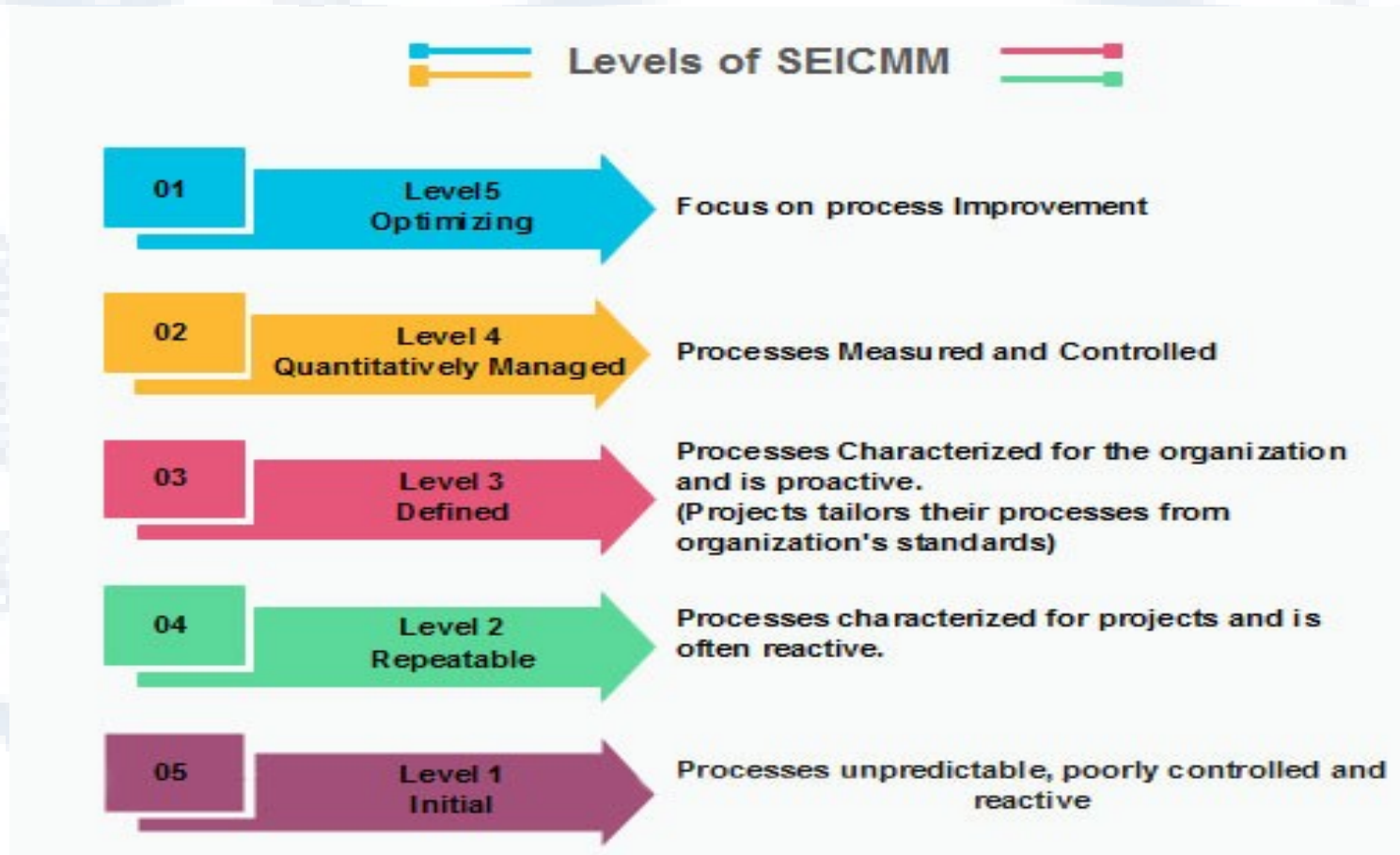
Challenges in software specification

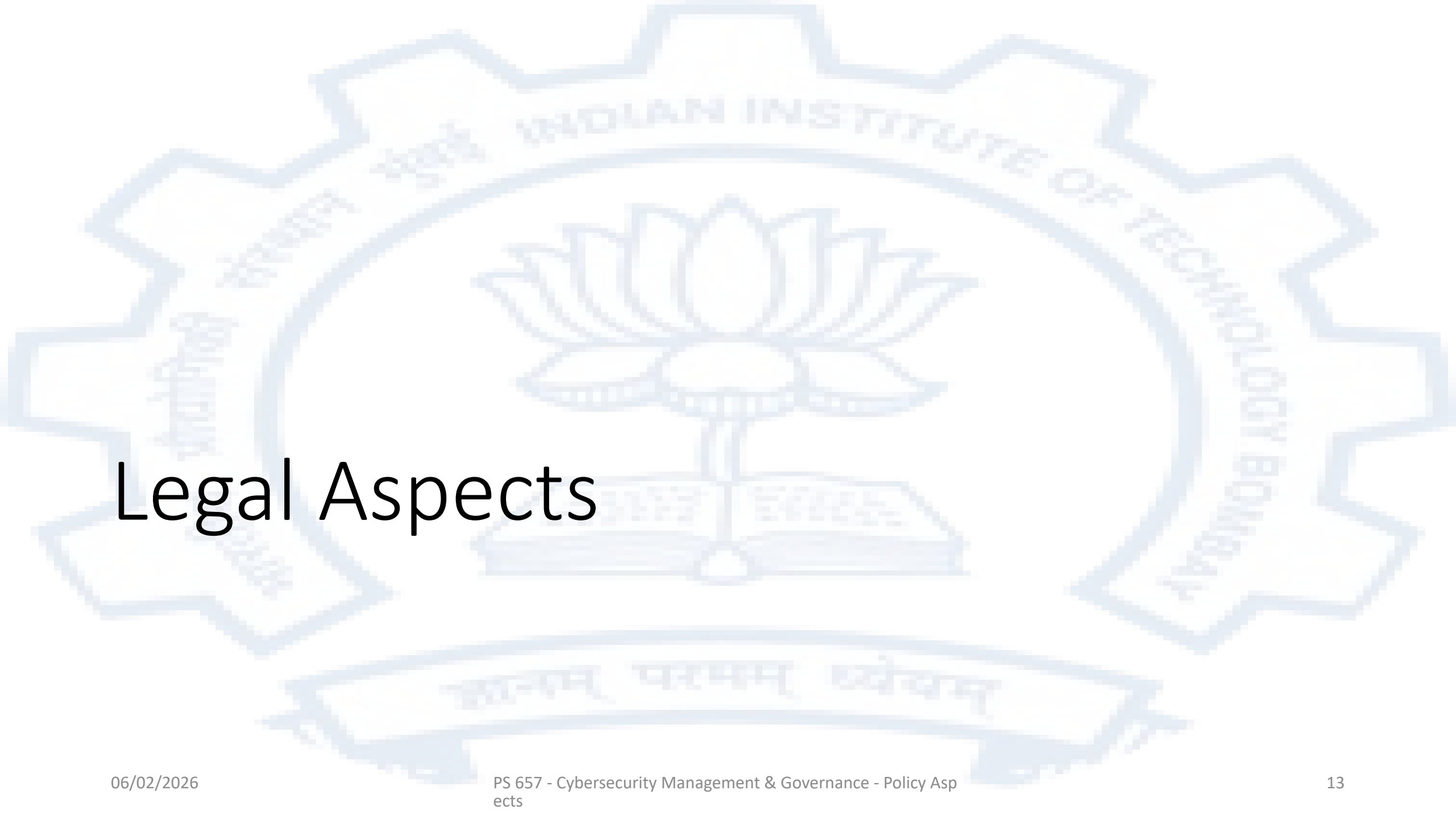
- Completeness of specification
 - Large software systems can have hundreds of high level requirements
 - Requirements can evolve or change
 - How are all requirements to be elicited?
 - Traceability
 - Does the design address all requirements?
 - Does the code implement all of the design?
- Formality of specification
 - Natural language v/s more formal mathematically oriented specification of requirements
 - Formal specification of design
 - Formal specification of code (other than the code is its own specification!!)

Practical Measures of Software Quality

- The available options are
 - Identify quality characteristics and attributes and try to specify ranges and measure them
 - Characterize the process of development of software without actually making an attribute based assessment
- The usual practice is to assess the process of software development and measure the “defect” density by some sort of testing
- One of the most widely used systems is the SEI-CMM system (Software Engineering Institute – Capability Maturity Model)

SEI CMM levels



The background of the slide features a large, light blue watermark of the Indian Institute of Technology Bombay (IITB) logo. The logo is a gear-like shape with a lotus flower in the center. The text "INDIAN INSTITUTE OF TECHNOLOGY BOMBAY" is written along the top inner edge of the gear. At the bottom, there is a banner with the Sanskrit motto "वैद्यमान् परममन् व्योचमान्".

Legal Aspects

Can software be considered as “goods”

- There is a difference of view in different countries on whether software can be considered goods and therefore be covered under product liability law.
- Even within a country the treatment may be different based upon the circumstances of the actual transaction.
- Software is most often licensed where the owner grants the purchaser the right to use it but not to resell or modify it. Under these circumstances the transaction does not qualify as sales of goods
- In some countries if software is embedded in another product, it is treated as goods

No Warranties – A typical example

- **DISCLAIMER OF WARRANTY.** THE SOFTWARE IS LICENSED "AS IS." YOU BEAR THE RISK OF USING IT. MICROSOFT GIVES NO EXPRESS WARRANTIES, GUARANTEES, OR CONDITIONS. TO THE EXTENT PERMITTED UNDER APPLICABLE LAWS, MICROSOFT EXCLUDES ALL IMPLIED WARRANTIES, INCLUDING MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT.
- **LIMITATION ON AND EXCLUSION OF DAMAGES.** IF YOU HAVE ANY BASIS FOR RECOVERING DAMAGES DESPITE THE PRECEDING DISCLAIMER OF WARRANTY, YOU CAN RECOVER FROM MICROSOFT AND ITS SUPPLIERS ONLY DIRECT DAMAGES UP TO U.S. \$5.00. YOU CANNOT RECOVER ANY OTHER DAMAGES, INCLUDING CONSEQUENTIAL, LOST PROFITS, SPECIAL, INDIRECT OR INCIDENTAL DAMAGES.

<https://www.microsoft.com/en-us/windowsvirtualdesktop/eula>

Aspects of Software Security

- Security features implemented by software such as cryptography, authentication, access control etc.
- Security issues arising from **vulnerabilities** in the software that allow software to be used in unintended ways

Anecdotaly

Software Quality asks the question *does the software do what it is supposed to do?* while Software Security asks the question *does the software do *only* what it is supposed to do?*

Categories of Software Vulnerabilities

- Memory management vulnerabilities
- Structured output generation vulnerabilities
- Race condition vulnerabilities
- API vulnerabilities
- Side Channel vulnerabilities

Memory Management Vulnerabilities

The OS or language runtime or user programs often allocate and deallocate memory. These implementations may have bugs. These bugs can broadly be classified as

- Spatial – a program has access to areas of the memory that it should not (buffer overflows)
- Temporal – a program can access memory that it once had access to but is now in use by another program (dangling pointer)

Exploiting Memory Management Vulnerabilities

To exploit a memory management vulnerability, an attacker provides input that causes the target software to trigger the vulnerability and access chosen memory locations. The following are important attack categories

- Code corruption attacks – modifies program code to attacker code
- Control flow hijack – could possibly modify locations that contain return addresses
- Data only attacks – only data is modified. This data could be configuration settings or privileges
- Information leak attacks – the objective is only to read and exfiltrate the data

Structured output generation vulnerabilities

Many times a software component prepares structured output which is then considered as input to another process. Tampering carefully with this can result in vulnerability

```
>>>
>>> print(name)
User1'--
>>> print(password)
xyz@abc
>>> query = "select * from users where nm='" + name + "' + "and pw = '" + password + "'"
>>> print(query)
select * from users where nm='User1'-- ' and pw = ' xyz@abc '
>>> |
```

- The characters – start a comment in SQL
- If the intention of the query is to fetch all records with a given name and password then the use of the constructed user name User1'– causes the password match part of the where clause to be omitted

Most common structured output generation vulnerabilities

- SQL injection vulnerabilities
- Command injection vulnerabilities
- Script injection vulnerabilities

Other types of vulnerabilities

- Race condition vulnerabilities
 - Vulnerabilities arising out of incorrect handling of concurrency
- API vulnerabilities
 - Improper authentication of caller, improper error handling
- Side channel vulnerabilities
 - Timing and power consumption may leak information about a computation

Preventing Vulnerabilities

- Programming language design features
 - Eg. strong type systems
- Race condition prevention
 - Eg. rigorous management of concurrency
 - Assertions, contracts and defensive programming
- Secure software development practices
 - Assertions and contracts
 - Trapping all exceptions
 - Proactive defensive rules

Detection of vulnerabilities

- Manual Code Inspection
- Automated vulnerability detection
 - Static detection
 - Examines the code
 - Attempts to detect all vulnerabilities in a few passes
 - Dynamic detection
 - Executes the code and monitors the executions
 - Important aspects are
 - How should the execution be monitored
 - How many times should the code be executed

Mitigating exploitation of vulnerabilities

- Runtime detection of attacks
 - Runtime monitoring tries to identify anomalous behavior or blocks access to certain resources based on the context.
- Automated software diversity
 - Address Space Layout Randomization (ASLR) is one example
- Limiting privileges
 - Sandboxing, compartmentalization, whitelisting
- Software integrity checking
 - Trusted booting is an example

Web applications – client side vulnerabilities

- Phishing
 - Users directed to fake sites
- Clickjacking
 - Create an overlay that tricks users into clicking on something other than what they think they have clicked on
- Physical attacks
 - Smudge attacks
 - Shoulder surfing attacks

Web applications – server side vulnerabilities

- Injection
 - SQL injection
 - Command injection
 - User uploaded files
 - Cross site scripting
- Misconfigured components
 - Firewalls
 - Load balancers
 - Databases

Standardization of vulnerability reporting

CVE – Common Vulnerabilities and Exposures database

Each CVE entry has the following format

- CVE ID number with four or more digits in the sequence number portion of the ID (e.g., "CVE-1999-0067", "CVE-2014-12345", "CVE-2016-7654321").
- Brief description of the security vulnerability.
- Any pertinent references (i.e., vulnerability reports and advisories).

A CVE Example

CVE-2021-44228 Detail

MODIFIED

This vulnerability has been modified since it was last analyzed by the NVD. It is awaiting reanalysis which may result in further changes to the information provided.

Description

Apache Log4j2 2.0-beta9 through 2.15.0 (excluding security releases 2.12.2, 2.12.3, and 2.3.1) JNDI features used in configuration, log messages, and parameters do not protect against attacker controlled LDAP and other JNDI related endpoints. An attacker who can control log messages or log message parameters can execute arbitrary code loaded from LDAP servers when message lookup substitution is enabled. From log4j 2.15.0, this behavior has been disabled by default. From version 2.16.0 (along with 2.12.2, 2.12.3, and 2.3.1), this functionality has been completely removed. Note that this vulnerability is specific to log4j-core and does not affect log4net, log4cxx, or other Apache Logging Services projects.

CVE Example- contd

References to Advisories, Solutions, and Tools

By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

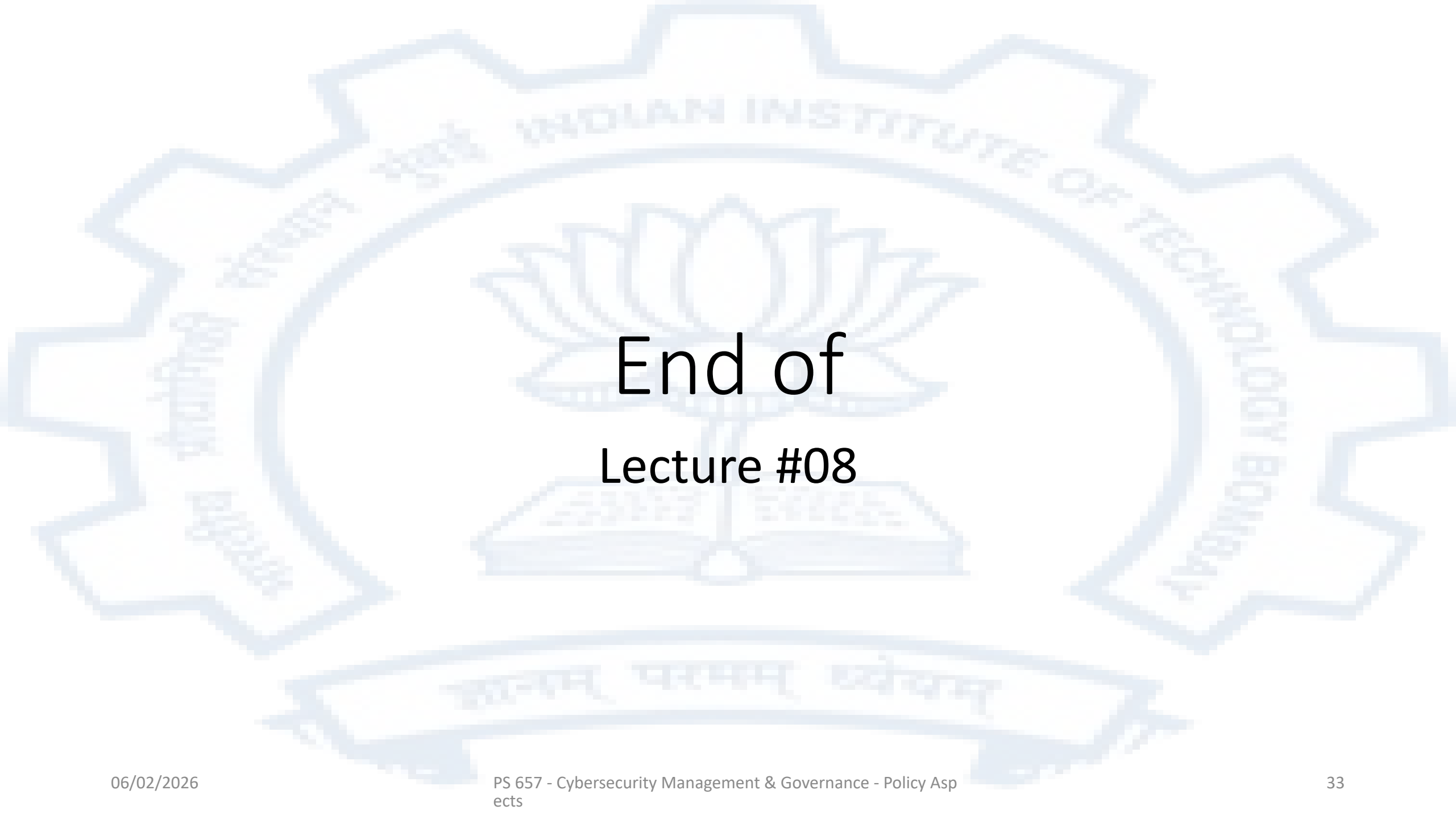
Hyperlink	Resource
http://packetstormsecurity.com/files/165225/Apache-Log4j2-2.14.1-Remote-Code-Execution.html	Third Party Advisory VDB Entry
http://packetstormsecurity.com/files/165225/Apache-Log4j2-2.14.1-Remote-Code-Execution.html	Third Party Advisory VDB Entry
http://packetstormsecurity.com/files/165260/VMware-Security-Advisory-2021-0028.html	Third Party Advisory VDB Entry
http://packetstormsecurity.com/files/165260/VMware-Security-Advisory-2021-0028.html	Third Party Advisory VDB Entry
http://packetstormsecurity.com/files/165261/Apache-Log4j2-2.14.1-Information-Disclosure.html	Exploit Third Party Advisory VDB Entry

To summarize

- Software development is complex and error prone
- We now use ecosystems that consist of thousands of software components
- It is very difficult to ensure that software is free of bugs and vulnerabilities
- Software vulnerabilities can be exploited to allow malicious users to do things that they are not authorized to do

OWASP – Open Worldwide Application Security Project

- OWASP Top 10 – 10 most important vulnerabilities and risks (updated regularly) <https://owasp.org/www-project-top-ten/>
- OWASP Software Assurance Maturity Model
- OWASP Development Guide
- OWASP Testing Guide
- OWASP Webgoat – A deliberately insecure web application

The background of the slide features a large, light blue watermark of the Indian Institute of Technology Bombay (IIT Bombay) logo. The logo is a gear-like shape with a lotus flower in the center. The text "INDIAN INSTITUTE OF TECHNOLOGY BOMBAY" is written along the top inner edge of the gear, and "सत्यम् धर्मम् कर्म" is written along the bottom inner edge.

End of Lecture #08